KanoonBuddy: Tech Stack

Oct 6, 2026 · @Akshita

Summary

Version 1 runs on one AWS server in Mumbai, serves only the Supreme Court (37,014 cases) plus acts_india, and includes upload-and-chat (built in v1, switched on for users only if the interviews and the legal opinion support it). You build it alone with AI coding agents, so the stack favours ready-made services and boring, well-known tools that an AI writes well.

What v1 does: research chat with checked citations, Act and section lookup, upload and chat with OCR, free public Act pages and Supreme Court outcome-statistics pages, a usage meter, login and payments, and organisation accounts for the Chambers plan.

How it is built: a fixed step-by-step pipeline (not a team of free-roaming agents) in Python, a Next.js website, Qdrant for RAG search with Voyage embeddings, Jev for fast checks and decisions, Gemini for writing answers, OKF knowledge notes for checked background facts, PostgreSQL for app data, Redis for jobs and caching, S3 for files, Firebase for login, Razorpay for payments, Langfuse for watching AI calls.

The one rule: every outside service (AI model, Jev, embeddings, OCR, login, storage, payments) sits behind its own small adapter file. A later engineer can swap any one part without touching the rest. That is what keeps the stack changeable instead of static.

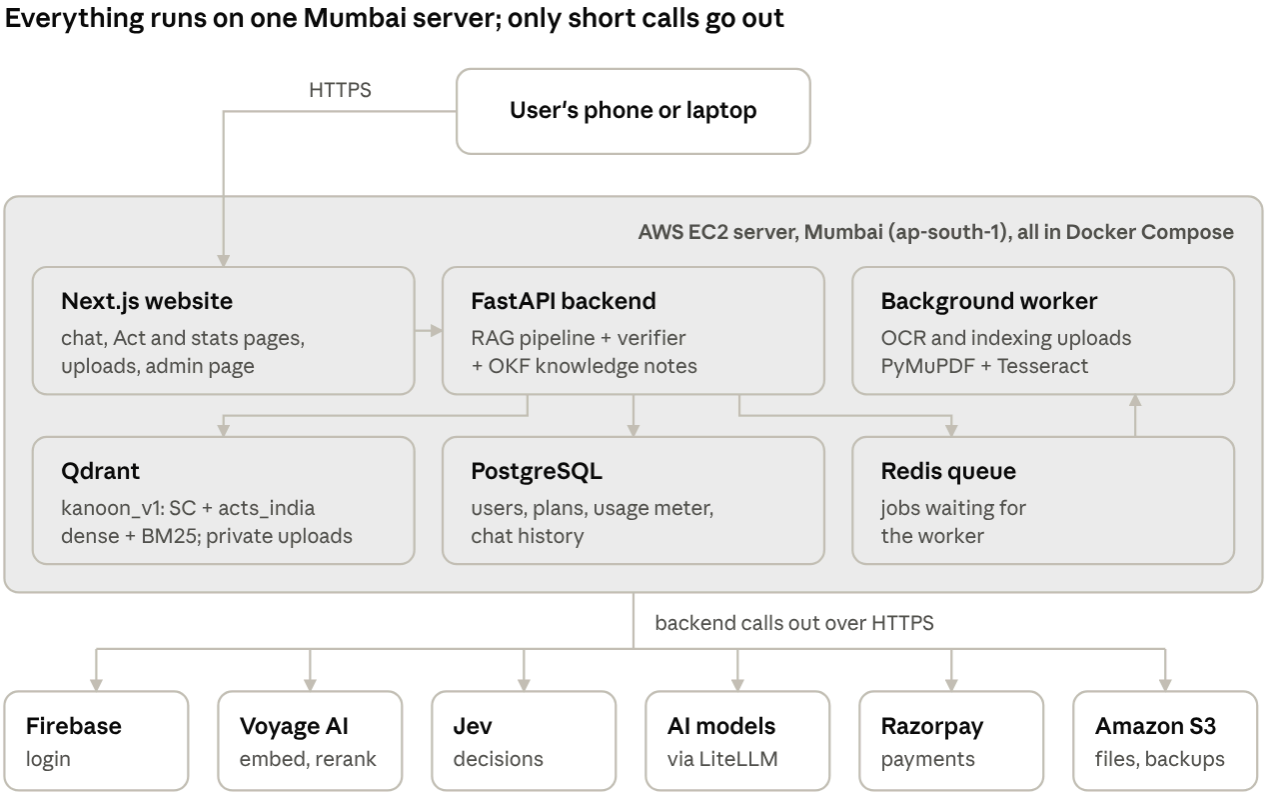
Rough running cost at launch: about ₹7,200 to ₹7,500 a month, most of it the server and its disk. See Running cost.

Labels used, matching the PRD: **Measured** = checked on your local copy of the dataset (from the PRD's audit). **Dataset card** = claimed by the dataset's own pages. **Checked** = confirmed on a public page on the date given; when that page is a vendor's own price or claim, the PRD calls it a **Competitor claim**. **Estimate** = my arithmetic; replace with measured numbers. **Needs verification** = not confirmed, with how to confirm it.

Revised 9 October 2026 after a full PRD review, applying final decisions D1 to D6. See the Change log at the end. Owner: Akshita Johari (founder).

Name: KanoonBuddy is a working name only. The PRD recommends Praman, pending a trademark search. The product name and the collection name each sit in one config value (for example PRODUCT_NAME and CORPUS_COLLECTION), so a final brand needs a config change, not a code change.

Architecture



KanoonBuddy v1 architecture · one server, six outside services

Six parts run in Docker on one EC2 server in Mumbai; the six services below it are each reached through one adapter file. Langfuse and Sentry are two more outside services, for monitoring only (see Where data goes). The OKF knowledge notes are Markdown files inside the backend's code, so they need no separate box or server. Uploads go from the browser straight to S3 with short-lived links, and the worker reads them from there.

How one research answer is produced:

1. **Remember.** The backend loads the last 6 turns of the conversation in full, plus a short running summary of older turns (stored on the conversation, refreshed by the cheap model every few turns).
2. **Rewrite.** The cheap model turns the latest message plus that context into one standalone search query, and pulls out any Act, section, case number or year. This is what makes short follow-ups like "what about limitation?" work.
3. **Decide (Jev).** At the same time: scope check, filters, safety check, and which knowledge notes apply. Easy-or-hard routing is switched off at launch: every question goes to its plan's default model until the stronger model is chosen. An out-of-scope question stops here with the coverage message.

4. **Check the cache.** If the same rewritten query was answered recently, return the saved answer from Redis. No AI call, no charge to the user's limit.
5. **Search.** Named sections or case numbers go to an exact payload filter. Everything else runs meaning search (voyage-4) and keyword search (BM25) together in Qdrant, using Jev's filters, and merges the top 50.
6. **Re-rank.** Voyage rerank-2.5-lite keeps the best 10 passages.
7. **Gate (Jev).** Each passage is rated answers it / partly / not relevant. Not-relevant passages are dropped; if none is good enough, the user gets "not found" and nothing is written. At most 8 of the rest go on, each with a short source ID for this turn (S1 to S8).
8. **Answer.** The chosen model writes using only those passages and any matching reviewed knowledge notes (given to it separately as background notes) and must put a source ID after every point it takes from one.
9. **Verify.** The free checks run first, then Jev's claim-support check on each sentence and its source (see Citation verifier). Failures are removed and logged.
10. **Show.** The answer text streams word by word. The checked citations arrive as one final message after verification, with a tick or warning each, links to pdf_url or source_url, the data-as-of date, the CC BY credit ("Data: Open India Law by Vaquill AI, CC BY 4.0") and the not-legal-advice notice. So the user never sees a citation appear and then vanish.
11. **Save.** The message, the source IDs used, Jev's decisions and confidence, the model name and the token counts are saved, and the usage meter counts one answer.

Stack by layer

Each row is one part of the system. "Swap to later" is what an engineer can replace it with without rewriting the rest.

Layer	v1 choice	Why, in plain words	Swap to later
Website (frontend)	Next.js + TypeScript + Tailwind, built as an installable web app (PWA)	One codebase for phone and desktop. AI coding tools write it very well. Server-rendered pages load fast and show up on Google	Any React framework; a native app if interviews demand it

Layer	v1 choice	Why, in plain words	Swap to later
Backend API	Python + FastAPI	Python has the best libraries for AI, PDFs, OCR and DuckDB. FastAPI is simple and fast	Split into separate services when traffic grows
Background jobs	Redis + ARQ (a small Python job queue); Redis also holds the answer cache	OCR and indexing an upload take seconds to minutes; the user should not wait on a frozen screen. The worker runs one job at a time, capped at 1 CPU, so chat stays fast	AWS SQS + Lambda workers
Vector search	Qdrant (self-hosted, Docker) with one new collection: Supreme Court + acts_india only	You already have the vectors in Qdrant. A Supreme Court-only copy is small enough for one cheap server	Qdrant Cloud, or a bigger server as courts are added
Keyword search	Qdrant sparse vectors (BM25) in the same collection	Exact words, section numbers and case numbers. Lives next to the semantic vectors, so one query does both. Decided: this settles the PRD's open choice between PostgreSQL full-text search and Qdrant sparse vectors	OpenSearch, if keyword search grows complex
Query embeddings	Voyage voyage-4	Must match the stored vectors. $0.06 per million tokens, first 200M tokens free (Checked)	Only by re-embedding everything; keep it behind one adapter

Layer	v1 choice	Why, in plain words	Swap to later
Re-ranker	Voyage rerank-2.5-lite; backup: open-source bge-reranker on the server	Same vendor as embeddings. $0.02 per million tokens, first 200M free (Checked). Backup runs if Voyage is down	Any reranker behind the same adapter
AI model gateway	LiteLLM (Python library)	One way to call Gemini, Claude, GPT, Kimi and others. Changing model = changing one line of config	OpenRouter or a hosted gateway
AI models	Free: Gemini 2.5 Flash-Lite (paid tier). Student and Pro: Gemini 3.1 Flash-Lite. A stronger model later for hard Pro questions. See AI models by tier	Cheapest reliable option that does not train on user data	Any model the gateway supports
Decision layer (Jev)	Jev by TypeSafe AI (early access)	Fast, cheap fixed-choice decisions with a confidence score: scope check, filters, easy-or-hard routing (off at launch), answer-or-refuse gate, claim-support check, safety check. See Decision layer: Jev	Gemini Flash-Lite with fixed choices (the built-in fallback)
Knowledge notes (OKF)	OKF-style Markdown files in the code (knowledge/ folder), advocate-reviewed	Checked facts search cannot get right alone, such as IPC → BNS. Sits beside RAG search. See Knowledge notes (OKF)	A database table or a graph store, if notes grow into thousands

Layer	v1 choice	Why, in plain words	Swap to later
App database	PostgreSQL 16 in Docker on the same server, daily backups to S3	Users, plans, usage meter, chat history, upload records	AWS RDS when you want managed backups
File storage	Amazon S3 (Mumbai), encrypted, auto-delete rules	Uploaded files and backups. Cheap and private by default	No reason to change
OCR (reading scans)	PyMuPDF for PDFs with text; Tesseract (English + Hindi) for scans	Free and runs on your server, so client documents never leave it	AWS Textract for hard scans, if quality needs it
Login	Firebase Authentication (email + Google sign-in)	Free up to 50,000 monthly active users (Checked). No login code to write	Supabase Auth or AWS Cognito; see Login and payments
Payments	Razorpay subscriptions	UPI, cards, Indian GST invoices	No reason to change
Hosting	One AWS EC2 server in Mumbai (ap-south-1) running everything in Docker Compose	Cheapest setup that a solo builder can run. Data stays in India	Split into services; managed database and Qdrant
Errors and monitoring	Sentry (free plan) + Langfuse (free cloud or self-hosted)	Sentry catches crashes. Langfuse records every AI call: prompt, answer, cost, speed	Datadog or Grafana later
Code and deploy	GitHub + GitHub Actions	Every push runs tests, then deploys to the server	Same

Agents and frameworks

Honest recommendation: do not use CrewAI or a multi-agent orchestrator inside the product for v1. Use agents to *build* the product, and use a fixed pipeline to *run* it.

Why: a research answer always follows the same 11 steps (remember, rewrite, decide, cache, search, re-rank, gate, answer, verify, show, save). Jev's checks are fixed steps in that list, not agents choosing what to do. A team of agents that decide for themselves which step to run is slower, costs more per answer, and is harder to debug. When an agent picks the wrong step, a lawyer gets a wrong answer and you cannot easily see why. Your PRD's trust promise depends on every answer going through the citation verifier, every time. A fixed pipeline guarantees that.

Where	Use	Why
Building the code	AI coding agents (Claude Code, Cursor or similar)	This is where agents save you the most time. You describe a feature, the agent writes and tests it, a second agent reviews it
Running research chat	Plain Python pipeline (fixed steps, no framework)	Predictable, cheap, easy to trace. Each step is a small function an engineer can replace
Calling AI models	LiteLLM	Swap models without code changes
Later features with real choices (counter-arguments, moot court)	LangGraph, only if needed	LangGraph lets you draw the steps as a flowchart with fixed checkpoints, so the verifier can still be forced. Add it in month 6, not now
One-off data jobs (cleaning the Supreme Court disposition values, nightly checks)	A simple script with an AI model inside, run by hand or on a schedule	Batch work, no user waiting

Not recommended: CrewAI (built for open-ended agent teams, not a fixed legal pipeline), and full LangChain (many layers you would not use, and it changes often, which breaks code that an AI wrote last month). You can still use small pieces such as LangChain's text splitters if they help.

How to build with coding agents, solo:

1. Keep a file called `CLAUDE.md` (or `AGENTS.md`) in the code that explains the stack, folder layout and rules. Every coding agent reads it first.

2. Build one small feature at a time, each with tests. Ask the agent to run the tests before it says it is done.
3. Use a second agent, or a fresh session, to review each change against the rules in Keeping it changeable.
4. Never let a coding agent touch the production server directly. It pushes to GitHub; GitHub Actions deploys.

AI models by tier

Use a cheap paid model from day one instead of a free-tier model. The cheap paid models cost a few paise per answer, and free tiers come with a privacy problem.

The privacy problem, in plain words. It is not about your data. It is about your users' data. A lawyer's question or uploaded file may contain a client's name, facts of a case, or privileged advice. On Google's free Gemini API tier, prompts and answers can be used to improve Google's products; once billing is switched on, they are not (Google forum answer). Sending client material to a model that may train on it is exactly what the PRD's Security and Privilege sections say to avoid, and it would also break the promise "we never train on your data" on your privacy page.

Kimi K3 is not free. Its API costs $3 per million input tokens and $15 per million output tokens, and it always "thinks" before answering, which adds hidden output tokens (pricing). Only the Kimi.ai chat app is free, and you cannot call that from your product.

Plan	Task	Suggested model (v1)	Rough cost per answer (Estimate)
Free	Research chat, short answers	Gemini 2.5 Flash-Lite, paid tier: $0.10 in / $0.40 out per million tokens (pricing)	About ₹0.06
Free and paid	Query rewrite, conversation summary (classifying questions is Jev's job)	Same Flash-Lite model	Under ₹0.02
Student / Pro	Research chat, upload and chat	Gemini 3.1 Flash-Lite: $0.25 in / $1.50 out (pricing)	About ₹0.17

Plan	Task	Suggested model (v1)	Rough cost per answer (Estimate)
Pro (later)	Drafting, hard research	A stronger model (Claude, GPT, Gemini Flash or Kimi K3), picked after the week-1 test of 50 real questions	₹1 to ₹5

Cost-per-answer arithmetic: about 4,000 input tokens (the question plus 8 passages of about 500 tokens each) and 600 output tokens, at ₹88 per US dollar. Voyage embedding and re-ranking add about ₹0.05 per answer (re-ranking works out to about ₹0.044; rounded up to include the query embedding), and both are free for the first 200M tokens. Needs verification: is that allowance one-time or monthly? Measure real numbers in Langfuse during week 1.

Fallback: if Gemini fails or times out, LiteLLM retries on a second provider (for example a cheap open-weight model on OpenRouter or Groq, paid tier). Check that provider's data terms first. If both fail, the user sees the sources with links but no written answer, as the PRD says.

Every model used must be on terms that forbid training on your data. Write this into the adapter: the model list in config only accepts providers marked "no training".

Search: meaning + exact words

Every search runs three ways at once, and the results are merged. This is called hybrid search.

Kind of search	What it finds	How it works	Cost per query
Meaning (semantic)	"bail for economic offences" finds judgments that never use those exact words	Voyage voyage-4 turns the question into numbers; Qdrant finds the closest passages	One Voyage call
Exact words (keyword)	Rare words, party names, phrases in quotes	BM25 sparse vectors stored in the same Qdrant collection	Free
Exact lookup (filters)	"Section 438 CrPC", a case number, an SCR citation	Qdrant payload filters on indexed fields (section_number, citation, case_number)	Free

Qdrant's query API merges the meaning and keyword results in one call (a method called Reciprocal Rank Fusion). Exact lookups skip the merge and go straight to the record. Then the re-ranker keeps the best 10 passages, and Jev's gate drops any that do not help, keeping at most 8 (S1 to S8) for the answer. So: search merges the top 50, re-rank keeps 10, the gate keeps at most 8.

Decided: keyword search uses BM25 inside Qdrant. This settles the PRD's open choice (PostgreSQL full-text search or Qdrant sparse vectors). It also covers the PRD's failure state "Voyage API is down": keyword search and exact lookups keep working, labelled "keyword results only".

Building the v1 search collection

Your local Qdrant holds all 32.5M vectors. v1 needs only the Supreme Court and acts_india, so you make one new, small collection and ship only that:

1. On your own machine, read Supreme Court points out of legal_corpus_v1 and v2 with a court filter, with their vectors (Needs verification: the exact court field name and value in the payload).
2. Read the matching chunk text from the parquet files with DuckDB and compute BM25 sparse vectors for each (fastembed library, free, runs locally).
3. Write both into a new collection, kanoon_v1, with two named vectors: dense (the existing voyage-4 vectors, no re-embedding) and sparse (BM25). Copy acts_india in the same way.
4. Turn on scalar quantization so the vectors use about a quarter of the memory, and keep the full vectors on disk.
5. Take a Qdrant snapshot, upload it to S3, restore it on the AWS server.

Size (Estimate): acts_india is about 1.1M points. The Supreme Court point count is not yet measured; count it in step 1. At about 13.5 KB per point (PRD arithmetic), acts_india plus 2 to 3M Supreme Court points need roughly 42 to 55 GB of disk and, with scalar quantization, about 4 to 5 GB of memory.

Citation verifier

Mostly plain Python, with one Jev check. After the model finishes writing, the verifier checks every source ID in the answer. A citation passes only if all four checks pass:

1. **Retrieved for this question.** The source ID was one of the passages given to the model this turn (S1 to S8). A real case the model remembered from training, but that was not retrieved, fails. This stops the model citing a real but irrelevant case. (Free, Python.)
2. **Record exists.** That passage's record ID exists in kanoon_v1. (Free lookup, Python.)
3. **Quotes match.** Any words in quotation marks appear in that record's text, exactly, after tidying spaces and punctuation. (Free, Python.)
4. **Claim supported.** Jev rates whether the cited passage supports the sentence it is attached to: supports (tick), partly (warning mark), does not support (fails). In upload chats this check uses Gemini (paid tier) instead, because upload text never reaches Jev.

Display note: for Supreme Court cases, the citation string (for example an SCR reference) is shown exactly as stored, labelled "as recorded in the dataset", and never retyped by the AI. It is filled for 100% of Supreme Court chunks (Measured).

A failed citation is removed from the answer, and the rest of the answer shows a note that one authority could not be verified. It is also written to an **invented-citation log** (question, model, the failed citation, the reason). The rate of answers with a removed citation, for any of the four reasons (not retrieved, record missing, quote not found, claim not supported), is a launch metric: target under 0.5% of answers, shown on the admin page and checked on every release with the 200-question test set.

What the tick means, and what it does not

The tick is the core trust claim, so it states only what the checks can prove. Suggested wording, from the PRD: **"Record found in dataset"**, with the data-as-of date and a short link to this definition. Ticks are shown per citation, not one badge for the whole answer, as the PRD recommends.

The tick means	The tick does NOT mean
The cited case or Act section exists as a record in the dataset, matched by its record ID	The case is good law. It may have been overruled, distinguished or reversed on appeal
Words in quotation marks appear in that record's text	The quoted passage is the holding, or the AI's summary of it is right
The link goes to the record's pdf_url (judgments) or source_url (Acts)	The link works today, or the text is current law. The data is a snapshot (v2026.08)
A citation string, if stored, is shown exactly as recorded, labelled "as recorded in the dataset"	The citation string is correct or complete. It is filled for 100% of Supreme Court chunks and 0% of High Court chunks (Measured), so High Court answers will show none

Beyond the PRD: the Jev claim-support check. The PRD's verifier checks that a record exists and quotes match. This doc adds a fourth check (Jev rates whether the passage supports the sentence). It narrows, but does not remove, the "summary is right" gap above: a passage can support a sentence and the case can still be bad law.

Caching repeated questions

Lawyers ask about the same well-known sections again and again, so v1 caches in Redis:

- **Exact match:** the rewritten query, plus the filters, plus the model name, is the key. Saved for 7 days (Estimate; tune from real use).
- Only answers that passed verification are cached. Upload chats are never cached, because they are private.
- Clear the cache when the data refresh adds new judgments.

A similar-meaning cache and the model provider's own prompt caching can come later, once the cost dashboard shows they are worth it.

An admin screen on top of Qdrant

Two layers, both behind your own admin login:

- **Qdrant's built-in Web UI** (comes free with Qdrant at /dashboard). Browse collections, look at points and run test searches. Open it only through a private tunnel, never on the public internet.
- **A small admin page in the Next.js app**: search the corpus the way a user would, see the raw scores from each of the three searches, open any record's payload, and mark bad records. This is also where you will run the 200-question test set.

Decision layer: Jev

Jev makes every fixed-choice decision in the pipeline: is the question in scope, which filters to use, do the passages answer it, and does each cited passage support the sentence it is attached to. Gemini still writes the answers, because Jev does not write text.

What Jev is. A new kind of AI model from TypeSafe AI. You give it information plus a fixed set of choices, and it returns one choice with a confidence score that the maker says is honest ("calibrated"). It cannot return anything outside the choices you set. Vendor claims: 70 to 500 ms per call, $0.042 per million input tokens, output free, up to 255 choices per question (TypeSafe blog). Released in early access on 15 September 2026.

Where Jev runs in v1

Step	Question Jev answers	Choices	What happens next
Scope check (before search)	Can KanoonBuddy answer this?	in scope, needs a High Court, needs a tribunal, after the snapshot date, not a legal question	Anything but "in scope" shows the PRD's coverage message. No search, no writing cost
Filters (before search)	Which court, years and Act does this question point to?	Court and year range from fixed lists; Act chosen from up to 20 candidates found by keyword match first	Filters sent to Qdrant, so search is narrower and more accurate
Model routing	Is this easy or hard?	easy, hard	Switched OFF at launch: every question goes to its plan's default model. Turns on only after the stronger model is chosen; then hard Pro questions go to it

Step	Question Jev answers	Choices	What happens next
Answer-or-refuse gate (after re-rank)	Does each passage help answer the question?	answers it, partly, not relevant	If no passage clears the threshold (start at 0.6, Estimate), the user gets "not found" and no answer is written. Passages marked not relevant are dropped
Claim-support check (after writing)	Does this cited passage support this sentence?	supports, partly, does not support	"Does not support" removes the citation and logs it. "Partly" shows a warning mark instead of a tick
Safety check	What kind of request is this?	normal, asking for personal legal advice, trying to trick the AI	Personal advice gets a stronger disclaimer; tricks are blocked
Uploads	What is this document, and is the OCR readable?	Document types (bail application, agreement, notice, judgment, other); readable or not	Better answers on uploads; unreadable files fail early and are not charged. Upload text never goes to Jev: in upload chats, Gemini (paid tier) makes this call

Scope, filters, routing and safety run at the same time as the Voyage embedding, so they add little waiting. Gate and claim checks run on all passages at once (in parallel).

Offline jobs (no user waiting)

- **Cleaning the Supreme Court's disposition values** into a short list (allowed, partly allowed, dismissed, disposed of, other), and labelling each as final outcome or interim order. The Supreme Court has about ten main values (Measured). Humans then check the 50-judgment Supreme Court sample instead of labelling from scratch. The 577 High Court values wait for Phase 2.
- **Spotting junk** in the judges and party-name fields, so only real names are ever shown.
- **Sorting wrong-answer reports** by type, so the advocate advisor sees the serious ones first.

Later features

Case analysis ("favours applicant / respondent / neutral", confidence), counter-argument strength, grading moot court answers and quizzes. These are already fixed-choice fields in the system architecture's JSON plans.

Safeguards

- **Own adapter and switches.** Jev is called from one file, `adapters/decisions.py`. Each use above has its own on/off switch in config, so one can be turned off without touching the others.
- **Fallback.** If Jev is down or slower than 800 ms, the same decision goes to Gemini Flash-Lite through LiteLLM with fixed choices. The three free verifier checks (retrieved for this question, record exists, quote matches) always run, Jev or not.
- **Thresholds from data.** Every Jev decision and its confidence is logged in Langfuse. Set each threshold from the 200-question test set, and turn each use on for users once it beats the version without it.
- **Data location.** Jev's servers are on the US West Coast, so questions leave India and each call adds network time from Mumbai. Upload text never reaches Jev: in upload chats, the claim check and any decision that would include upload text use Gemini (paid tier) instead.
- **Access.** Jev is in early access: apply now so access is ready by build step 2.

Cost: about ₹0.03 per answer (Estimate: about 8,000 input tokens across all checks at $0.042 per million). The offline cleaning jobs cost a few hundred rupees in total for the Supreme Court.

Knowledge notes (OKF)

A small set of checked Markdown notes sits beside RAG and carries the facts that search cannot get right on its own, starting with which old law maps to which new law. RAG still searches the judgments; the notes add curated background.

What OKF is. The Open Knowledge Format is an open standard from Google Cloud for giving AI curated knowledge as plain Markdown files, linked to each other like a wiki, instead of as vectors in a database. Version 0.1 was published on 12 June 2026, and Google calls it a starting point, not a finished standard (Context Studios, Mayhem Code). KanoonBuddy uses it simply as a folder of Markdown files in the code, so no OKF tool is required.

Why it matters here. In the PRD's hands-on test, even Prism said Section 377 IPC "remains active" without noting the BNS replacement. Old judgments do not know the law changed, so search alone cannot fix this. A note mapping IPC sections to BNS sections

can.

RAG and notes: who does what

	Knowledge notes (OKF)	RAG search
Holds	Facts you have checked: law maps, leading cases, limitation periods, coverage rules	All Supreme Court judgments and acts_india
Size	A few hundred files	Millions of chunks
How it is found	Jev's decide step picks the notes that apply (for example, "this question names an IPC section")	Meaning + keyword search, re-rank, Jev gate
Shown to the AI as	"Background notes"	"Passages" S1 to S8

First notes to build

Note	Drafted by	Source	Checked by
IPC → BNS, CrPC → BNSS, Evidence Act → BSA maps (from 1 July 2024)	AI, as tables	Official correspondence tables and the new Acts in acts_india	Advocate
Limitation periods	Script	Schedule to the Limitation Act in acts_india	Advocate
Leading Supreme Court cases per topic	AI	A script extracts citation strings from each judgment's full text, matches them to the citation field, and counts the most-cited (Needs verification on 50 judgments)	Advocate
Disposition label meanings	Jev cleaning job	Your dataset	You
Court hierarchy and legal terms	AI	Act text and general knowledge	You
What KanoonBuddy covers	You	The PRD	You

How notes are made and kept correct

1. A script (written with an AI coding agent) drafts each note as Markdown. Every fact links to the record ID it came from.
2. A check script confirms every linked record exists and every quote matches. Failing drafts are fixed or dropped.
3. A person reviews: an advocate for law maps, leading cases and limitation periods; you for coverage rules and terms.
4. Each file's header has `status` (draft or reviewed), `reviewed_by`, `last_checked` and its source record IDs. **Only reviewed notes are used in live answers.**
5. After each data refresh, the check script flags notes whose source records changed, and only those go back for review.

Folder layout: `knowledge/index.md` lists every note; one folder per type (`law-maps/`, `leading-cases/`, `limitation/`, `coverage/`, `terms/`); each note links to related notes.

Trust rules

- A note never counts as a citation by itself. When the answer uses a note, it cites the judgment or section the note points to, and the citation verifier checks that record as usual.
- Notes are labelled as background to the AI, kept apart from the retrieved passages.
- A wrong note gives wrong answers confidently, so the advocate review cannot be skipped.

Cost: close to nothing. Notes are files in the code, read at startup, adding a few hundred tokens to an answer only when they apply. The real cost is the advocate's review time for the law maps (hundreds of rows).

Public pages and outcome statistics

Free, no-login pages for Act sections and Supreme Court outcome counts bring in users from Google search at almost no running cost. This is the PRD's main way to get the first 100 users. The pages use free database filters only, with no Voyage, Jev or AI writing call.

The pages

Page	Address (example)	Shows
Act section	/acts/{act}/{section}	Section text, in-force status, related knowledge notes (for example the old IPC section it replaced)
Outcome statistics	/stats/supreme-court/{year}	How Supreme Court cases that year were decided: allowed, partly allowed, dismissed, disposed of, other
Statistic detail	/stats/supreme-court/{year}/{outcome}	The count, with links to example judgments

Every page is built on the server by Next.js, so it loads fast and Google can read it. A **sitemap** (sitemap.xml) lists every public page for Google. Every page has a **share card** (the preview picture shown in WhatsApp and LinkedIn) with the source, the data-as-of date and the CC BY credit, plus WhatsApp and LinkedIn share buttons.

Where the numbers come from

- A Jev cleaning job maps each disposition value to a short **outcome_label** (allowed, partly allowed, dismissed, disposed of, other) and marks **is_final_outcome** (final result, or an interim order). The Supreme Court has about ten values; the 577-value problem belongs to the High Courts (Measured).
- Counts include only cases where is_final_outcome is true.
- "Issue" filters (by Act or section) come later, once a reliable link between cases and the sections they cite exists (Needs verification).

Rules

- **Supreme Court stats go live only after the 50-judgment Supreme Court hand check passes,** with the pass rule agreed before checking. High Court stats wait for their own check (Phase 2).
- **Descriptive only:** counts, never predictions or win chances.
- A **not-legal-advice notice**, the data-as-of date and the CC BY credit on every page and share card.
- **No advocate names** on any page or card (BCI Rule 36). Judges and party names stay hidden, because the PRD's audit found them unreliable.
- Needs verification: search demand for these pages (a keyword tool), and what Indian Kanoon and others already show.

Mobile and slow networks

Many advocates work from a phone on a weak connection, so the website is built text-first and installable (a PWA), with no native app in v1.

- **Text first:** answers stream as plain text; judgments open as text with paragraph anchors; the PDF opens only on tap, with its file size shown.
- **Resumable uploads:** large files upload in parts (S3 multipart upload), so a dropped connection resumes instead of starting over. The size limit is shown before upload.
- **Offline saved answers:** saved answers and statistics are kept on the phone and open without a connection.
- **Share links:** answers and statistics get short links with share cards that preview well in WhatsApp.
- **Page-weight budget:** a size and load-time limit per page, tested on a throttled connection before launch (Needs verification: set the numbers after the interviews show which phones and networks advocates use).

Upload and chat

Uploads are read on your own server and stored encrypted in S3; only the few passages needed for an answer go to the AI model. This keeps client documents as private as a solo build allows.

How an upload flows:

1. The browser asks the backend for an upload link, then uploads straight to S3 using that short-lived link (a "pre-signed URL"), so large files never pass through the API. Limits on size and page count are shown before upload (Needs verification: set the limits, for example 25 MB and 200 pages).
2. The backend creates an upload record and puts a job in the Redis queue. The user sees each status as it changes: **uploaded → extracting → indexing → ready**, or **failed** with a reason. When OCR is needed, the screen says so, because it takes longer.
3. The worker samples a few pages to see whether the file has a text layer. PDFs with text go through PyMuPDF and Word files through python-docx. Scans go through Tesseract OCR (English and Hindi). If OCR quality is too low (Needs verification: set the threshold on 20 real scans), the status is failed, the user is not charged, and nothing is guessed.
4. The text is cut into chunks. **Every chunk keeps its page number and its start and end character positions**, so clicking a citation can jump to and highlight the exact passage in the viewer.
5. Chunks are embedded with voyage-4 (see Embedding choice for uploads; a free local model is the fallback) into one shared private Qdrant collection (user_documents). Every chunk carries both user_id and document_id, and every search filters on both.
6. The user asks questions. Answers cite page numbers in their own file, and the answer clearly labels "your document says" separately from "Supreme Court judgments say" when it also pulls case law.

Keeping OCR from slowing chat. The worker runs in its own Docker container, takes **one job at a time**, and is capped at 1 CPU and a fixed memory limit. Chat requests always get the other CPU. Once uploads grow, move the worker to its own small server; nothing else changes.

When uploads go live: upload and chat is built at build step 7, but switched on for users only if the 20 interviews show advocates would upload client documents and the advocate's written legal opinion supports it (privilege, DPDP role). Until then it stays behind a switch, used only for testing.

Embedding choice for uploads

Uploads go into their own collection, so they do not have to use voyage-4 like the judgments.

	Free local model on our server	voyage-4
Where the text goes	Stays on our Mumbai server	Every chunk of the client's document goes to Voyage, outside India
Cost	Free	$0.06 per million tokens after the free 200M (Checked)
Quality	Good multilingual models exist (for example bge-m3); needs testing on legal text (Needs verification)	Strong, and proven on our judgments
Speed	Slower: runs on the worker's 1 CPU	Fast
Privilege and DPDP risk	Lower	Higher

Recommendation: voyage-4 first, with a free local model as the fallback. It is the same model as the judgments, fast, and proven. Before uploads are switched on, confirm Voyage's terms for upload text: no training, how long it is kept, and where it is processed (Needs verification). If the terms are not acceptable, or the legal opinion asks for client text to stay in India, use a free local model (for example bge-m3) behind the same adapter. Either way, only the few passages needed for an answer go to Gemini (paid tier, no training), and upload text never goes to Jev.

Security rules for uploads (launch requirements):

- S3 bucket: private, encrypted with a key in AWS KMS, no public links. Downloads use links that expire in minutes.
- **Real hard delete.** "Delete" removes the file from S3, its extracted text from PostgreSQL, and all its vectors from Qdrant (by document_id), in one job. The job logs success, and a nightly check finds any leftovers. Backups age out on their normal cycle.
- Auto-delete: a lifecycle rule deletes uploads after a fixed period (Needs verification: period agreed with counsel). An optional "delete when I close this chat" mode. The default is shown in the app, not only in the terms.
- **Every query on uploads goes through one shared data-access function** that always adds the user_id filter. No feature writes its own Qdrant or upload query. A missing filter would leak one lawyer's files to another.

- No staff access by default. The audit log records who opened or deleted what, never the document text.
- A warning before upload about privileged material, as the PRD requires.
- Upload text goes only to vendors on no-training terms: chunks to Voyage for embedding (terms Needs verification), only the needed passages to Gemini, and never to Jev.

Cost: S3 storage is about $0.025 per GB a month (Needs verification on the AWS price page); 1,000 users uploading 20 MB each is about 20 GB, well under ₹100 a month. Tesseract is free. Textract, if added later for hard scans, is charged per page.

Where data goes

Every outside service gets only what it needs, and upload text never reaches Jev, Langfuse or Sentry. This table is also the basis for the privacy page.

Service	What it receives	Where	Rule
AWS (EC2, S3)	Everything we store: accounts, chats, uploads, backups	Mumbai	Encrypted disks and buckets
Firebase	Email, name, sign-in	Google (outside India)	Login only; no chats or uploads
Razorpay	Payment details	India	Card and UPI details stay with Razorpay; we never see them
Voyage	Question text; upload chunks, if uploads use voyage-4	Outside India	No-training terms; upload terms Needs verification
Jev	Questions and retrieved judgment passages	US West Coast	Never upload text
Gemini	Question, passages, knowledge notes; in upload chats, the needed upload passages	Outside India	Paid tier only (no training)

Service	What it receives	Where	Rule
Langfuse	AI-call logs	Cloud or self-hosted	**Masked:** a masking function replaces upload text, emails and phone numbers with placeholders before sending; keep IDs, token counts, cost and speed
Sentry	Error reports	Cloud	**Scrubbed:** request bodies, tokens, emails and document text removed before sending

Needs verification: each vendor's retention terms and where it processes data, listed on the privacy page before launch.

Hindi (P1)

Hindi is P1: within 3 months of launch, moving to P0 if the 20 interviews show strong demand. Before promising it, test each layer:

Layer	Test	Pass mark
Meaning search (voyage-4)	30 Hindi questions against English judgments: do the right cases come back?	Needs verification: set against the English results
Keyword search (BM25)	Hindi keyword search needs a Hindi-aware tokenizer that splits Devanagari words correctly and handles Hinglish. Not a v1 blocker	Needs verification: which library works with Qdrant's sparse vectors
Answers (Gemini)	Answer in Hindi while citations and quotes stay in the judgment's original language	Advocate checks 20 answers
Knowledge notes	Hindi names for common legal terms	You review
OCR (Tesseract)	Hindi and English already installed	Keep as is

The interface language (Hindi-first or English-first) is still an open question from the PRD.

Login and payments

Use Firebase only for login, and keep all your app data in your own PostgreSQL. Then moving away from Firebase later is a one-file change, not a migration.

A correction on cost: for login alone, Firebase is not cheaper than Supabase at your size, because both are free. Firebase Authentication is free up to 50,000 monthly active users for email and social sign-in, then charged per user (Firebase docs). Phone OTP login is charged per SMS, so start with email and Google sign-in only.

How it fits together:

1. The website signs the user in with Firebase and gets a token.
2. Every API call sends that token. FastAPI checks it with the Firebase Admin SDK.
3. Your PostgreSQL has a users table keyed by the Firebase user ID. Plans, usage counts and chats all point to this table, never to Firebase directly.

When to move from Firebase to Supabase (or AWS Cognito): not at a user count. Move only if one of these happens:

- You pass 50,000 monthly active users and Firebase's per-user charge becomes a real cost.
- Chambers or law firms ask for company sign-in (SAML or SSO). Firebase includes only 50 such users free.
- An engineer wants login and database in one product with database-level access rules.

Until then, staying put is cheaper than moving. Firebase lets you export users, including password hashes, so a later move is possible.

Payments with Razorpay:

- Razorpay Subscriptions for Student, Pro and Chambers, monthly and yearly.
- Razorpay sends a webhook to your backend when a payment succeeds or fails; the backend updates the user's plan in PostgreSQL. Always verify the webhook signature.
- The usage meter is a counter in PostgreSQL, checked before every paid action. Every plan, Free included, has a monthly answer limit. Answers served from the cache are free and do not count.
- Fee: about 2% per payment on standard plans (Needs verification on Razorpay's pricing page). GST on your prices is still undecided in the PRD.

Running cost

Launch costs about ₹7,200 to ₹7,500 a month, and about 21 paying users at the PRD's blended ₹349 cover it. That is far below the PRD's earlier ₹12,000 to ₹25,000, because v1 serves only the Supreme Court and compresses the vectors.

All figures are monthly, at ₹88 per US dollar, before GST. "Launch" assumes about 400 users and 3,000 answers a month (the PRD's 3-month target). "Illustrative scale" assumes about 2,000 users and 30,000 answers a month. It is not a forecast; the current target is 400 users at 3 months after launch.

Item	Launch	Illustrative scale: 2,000 users	Basis
Server: AWS EC2, Mumbai	₹4,820 (r7g.large: 2 CPUs, 16 GB memory)	₹9,650 (r7g.xlarge: 4 CPUs, 32 GB)	$0.0751 and $0.150 an hour on demand (r7g.large, r7g.xlarge)
Disk: EBS gp3	₹1,600 (200 GB)	₹1,600 (200 GB)	About $0.09 per GB (Needs verification). 200 GB holds the 42 to 55 GB collection, PostgreSQL, Docker images and working space; delete the snapshot file after restore
Public IP address	₹320	₹320	AWS charges for public IPv4 (Needs verification)
S3: uploads and backups	₹100	₹500	Estimate
AI model answers	₹200 to ₹500	₹2,000 to ₹5,000	₹0.06 to ₹0.17 per answer (see AI models by tier)
Jev decisions	₹100	₹900	About ₹0.03 per answer (see Decision layer: Jev)
Voyage embeddings + re-ranking	₹0	about ₹1,500	About ₹0.05 per answer after the free 200M tokens (pricing)
Firebase login	₹0	₹0	Free up to 50,000 monthly active users

Item	Launch	Illustrative scale: 2,000 users	Basis
Sentry, Langfuse, GitHub	₹0	₹0	Free plans
Domain	₹100	₹100	About ₹1,000 a year
Total	**about ₹7,200 to ₹7,500**	**about ₹16,600 to ₹19,600**	Plus Razorpay's fee on revenue

Ways to cut the launch bill:

- A 1-year reserved server brings r7g.large to about $0.052 an hour, roughly ₹3,350 a month instead of ₹4,820 (source). Commit only after the beta proves the server size is right.
- New AWS accounts and startup programmes (AWS Activate) often give free credits. Apply early (Needs verification: current credit amounts).

One-time costs (Estimate): building the kanoon_v1 collection runs on your own machine (₹0); uploading the 42 to 55 GB snapshot to S3 is free (AWS does not charge for data coming in). The PRD's legal opinion and security test are separate and still need quotes.

When the bill jumps: adding Bombay, Telangana and Kerala High Courts later means more points and memory. Count their points in your local Qdrant first, then size the server. Upgrade only when paying users cover the difference, as the PRD says. See Phase 2: High Courts.

Phase 2: High Courts

v1 serves only the Supreme Court and acts_india. The Bombay, Telangana and Kerala High Courts come next, and only when paying users cover their extra server cost (the PRD's trigger). These three were chosen in the PRD for archive depth: 45.0%, 45.4% and 39.9% of their cases predate 2015 (Measured). The PRD names Delhi as the next court after these, for practical demand.

Extra size and cost

The per-court point counts are not measured yet, so every figure here is an illustration until step 1 of Phase 2 counts them (Needs verification).

Item	How to work it out	Illustration: the PRD's 3M-point guess
Extra points	Count in your local Qdrant, filtered by court	3M (PRD assumption, for illustration only)
Extra disk	Points × about 13.5 KB (PRD arithmetic)	About 41 GB
Extra memory	Points × about 1 KB with quantization (1,024 numbers at 1 byte each, plus index)	About 3 GB (Estimate)
Server size	Total memory need vs 16 GB on r7g.large	Likely r7g.xlarge, 32 GB (Estimate)
Extra monthly cost	Bigger server plus extra disk	About +₹5,600: +₹4,830 server, + ₹800 for 100 GB more disk (Estimate)
Extra paying users needed	Extra cost ÷ ₹349 blended	About 16 more (Estimate)

What changes when High Courts are added

- **Full judgment text:** High Courts hold far more cases than the Supreme Court, so their full text may be too large for PostgreSQL. Serve it from the parquet files or S3 instead (Needs verification).
- **Citation strings:** filled for 0% of High Court chunks (Measured). The verifier falls back to record existence, quote matching and Jev's claim check.
- **Outcome statistics:** run the Jev cleaning job on the 577 High Court values, then the 50-judgment High Court hand check. High Court stats go live only if it passes.
- **Coverage:** update court_coverage (live, data-as-of date), the scope-check choices and the coverage message.
- **Junk fields:** run the judge and party-name junk check for these courts; keep them hidden if they fail.

Keeping it changeable

Seven rules let a future engineer replace any part without rewriting the rest. Put them in the `CLAUDE.md` file so every coding agent follows them.

1. **One adapter per outside service.** Each vendor (Voyage, Gemini, Jev, Firebase, S3, Razorpay, OCR) is called from exactly one file, such as `adapters/embeddings.py`. The rest of the code calls `embed(text)`, never the Voyage library directly.
2. **Settings live in config, not code.** Model names, API keys, limits and prices sit in environment variables and one settings file. Changing the AI model is a config change.
3. **The frontend talks to the backend only through the API.** FastAPI publishes an OpenAPI description automatically; the frontend's API client is generated from it. A new frontend team can start from that file.
4. **Database changes go through migrations.** Use Alembic, so every change to PostgreSQL tables is a numbered, reversible file.
5. **Everything runs in Docker.** One `docker compose up` starts the whole system on any laptop or server. Moving to separate servers later means moving containers, not rewriting code.
6. **Tests guard the trust promise.** The citation verifier and usage meter have tests that must pass before any deploy. The 200-question test set runs on every release.
7. **Each pipeline step is a separate function with a fixed input and output.** Rewrite, decide, search, re-rank, gate, write, verify and format can each be swapped or improved alone. The knowledge notes are plain files, so they can be edited without touching code.

Suggested folder layout:

```
kanoonbuddy/
  web/            Next.js website
  api/            FastAPI app (routes only)
  core/           pipeline steps: search, rerank, answer, verify
  adapters/       one file per outside service
  workers/        background jobs: OCR, indexing uploads
  data/           scripts to build kanoon_v1 from parquet + Qdrant
  knowledge/      OKF knowledge notes (reviewed Markdown files)
  evals/          200-question test set and scorer
  infra/          docker-compose, deploy scripts
  CLAUDE.md       rules for coding agents
```

Speed and reliability targets

The first word of an answer should appear within 3 seconds. A stream that starts fast feels fast, even if the full answer takes 15 seconds. Targets come from the system architecture doc; measure them in Langfuse from week 1.

Measure	Target
First word of a research answer	Under 3 seconds
Full research answer	Under 20 seconds
Search stage alone (search + re-rank)	Under 800 ms
Each Jev decision	Under 800 ms, or the Gemini fallback takes over
Act and section lookup, direct search	Under 500 ms
Upload ready: text PDF under 50 pages	Under 60 seconds
Upload ready: scanned PDF under 50 pages	Under 5 minutes
API availability	99.5% a month (about 3.6 hours of downtime)

What this means for one server: a single EC2 server has no backup if it fails. At launch, that is accepted to keep cost low. To reach 99.5%: turn on EC2 automatic recovery, keep daily snapshots in S3, and write down how to rebuild the server from them (target under 1 hour). Add a second server behind a load balancer only when paying users can cover it.

Watch first: search memory in Qdrant and OCR time. The architecture doc names these as the two things that break first.

Data model

v1 needs 17 PostgreSQL tables and 2 Qdrant collections. IDs are UUIDs and every table has created_at and updated_at. There is no password column anywhere: Firebase holds passwords, and your tables point to the Firebase user ID. Drafting and sandbox tables from the architecture doc are added when those features are built.

Accounts and billing

Table	Key fields
users	id, firebase_uid (unique), email, name, role (advocate, student, in-house, other), plan, status
subscriptions	id, user_id or organisation_id, plan, seats, razorpay_subscription_id, status, period_start, period_end

Table	Key fields
usage_daily	user_id, date, feature, requests, tokens_in, tokens_out, cost_estimate
organisations	id, name, owner_user_id, plan (chambers, ₹299 per user a month), seats (minimum 3)
memberships	organisation_id, user_id, role (admin or member), status (invited, active, removed)
consents	id, user_id, notice_version, purpose, given_at, withdrawn_at

Conversations and trust

Table	Key fields
conversations	id, user_id, module (research or document), document_id (if any), title, rolling_summary, message_count
messages	id, conversation_id, role, content, rewritten_query, model, tokens_in, tokens_out, latency_ms, from_cache, scope, route (easy or hard), safety, jev_used
message_sources	message_id, source_id (S1 to S8), record_id, chunk_id, rank, rerank_score, verified (yes or no), relevance (from the Jev gate), support (supports, partly, does not), support_confidence
invented_citation_log	id, message_id, query, model, cited_text, reason (not retrieved, record missing, quote not found, claim not supported)
feedback	message_id, user_id, rating, reason, comment

Uploads

Table	Key fields
documents	id, user_id, filename, mime_type, size_bytes, page_count, status (uploaded, extracting, indexing, ready, failed), fail_reason, ocr_used, storage_path, expires_at, deleted_at

Table	Key fields
document_chunks	id, document_id, page_no, char_start, char_end, text, vector_id

Corpus copy for fast reading (loaded once from parquet with DuckDB; the Supreme Court is small enough to sit in PostgreSQL)

Table	Key fields
cases	record_id, title, court, decision_date, case_number, citation, disposition (raw), outcome_label (allowed, partly allowed, dismissed, disposed of, other), is_final_outcome, outcome_checked (in the hand-check sample), pdf_url, full_text
act_sections	record_id, act_name, act_year, section_number, section_title, text, in_force, act_status, source_url
court_coverage	court, snapshot_version, data_as_of, points, live (yes or no), stats_live (yes or no)

Records

Table	Key fields
audit_log	id, user_id, action (login, upload, delete, export, admin view), target_id, ip, request_id, at. Never stores document text

Judges and party names are left out of display, because the PRD's audit found them unreliable.

Chambers organisations: an organisation groups 3 or more users under one bill. In v1, each member's chats and uploads stay private to that member; shared folders can come later. The shared data-access function still allows a read only if the user owns the item. **Data-as-of dates** are stored per court in court_coverage and shown on every result, stats page and Act page.

Qdrant collections

Collection	Vectors	Payload fields used as filters
kanoon_v1	dense (voyage-4) + sparse (BM25)	record_id, source_type (judgment or act), court, year, citation, case_number, act_name, section_number, in_force, act_status, outcome_label, is_final_outcome
user_documents	dense (voyage-4; free local model as the fallback)	user_id, document_id, page_no, char_start, char_end

API rules and audit log

All routes live under /api/v1, send and receive JSON, and need a Firebase token, except public Act pages. Chat answers stream using server-sent events (SSE): the text arrives word by word, then one final `citations` event after verification, then a `done` event.

v1 routes

Method and path	Purpose
GET /me	Profile, plan, answers left this month
GET /search	Direct search with filters, no AI answer (cheap)
GET /cases/{id}	Full judgment with paragraph anchors; backs every citation link
GET /acts/{act}/sections/{n}	One section; in-force status shown only when act_status is filled, otherwise labelled "in-force status not checked" (public, no login)
POST /conversations	Start a research or document conversation
GET /conversations	List, with cursor pagination
GET /conversations/{id}	Full history with sources
POST /conversations/{id}/messages	Ask a question; streams the answer
POST /documents/uploads	Get a pre-signed upload link and a document ID

Method and path	Purpose
GET /documents/{id}	Upload status
DELETE /documents/{id}	Hard delete: file, text and vectors
POST /feedback	Thumbs up or down plus a comment
POST /billing/webhook	Razorpay events (signature checked, no Firebase token)
GET /stats	Public outcome counts by court, year and outcome label (no login). Supreme Court only until Phase 2
GET /sitemap.xml	Lists every public Act and stats page for Google (served by the website)
POST /organisations	Create a Chambers organisation (minimum 3 seats, one bill)
POST and DELETE /organisations/{id}/members	Invite or remove a member
GET and POST /me/consents	See or withdraw consent (DPDP)
DELETE /me	Delete the account and all its data, including the Firebase user; cancels the Razorpay subscription. Payment and invoice records kept as the law requires (Needs counsel) (DPDP)
GET /me/export	Download a copy of my account data, chats and upload list (DPDP)

No /auth/signup or /auth/login routes: Firebase handles those in the browser.

Rules every route follows:

- **One error format:** `{"error": {"code": "quota_exceeded", "message": "...", "request_id": "..."}}`. The code is machine-readable so the website can show the right screen.
- **A request ID on every response**, also written into every log line and Langfuse trace, so one ID finds everything about a complaint.
- **Cursor pagination** on every list.
- **Limits in headers:** remaining answers and rate limits are returned as response headers.
- **The usage check runs before any AI call**, in the AI gateway module, and stops with a clear message at the limit instead of silently giving worse answers.

Audit log: logins, uploads, deletions, exports and any admin view of a user's data each write one row to audit_log (see Data model), with the user, action, target, IP and request ID. It never holds document text. How long to keep it needs a decision with counsel, as in the PRD.

Compliance

Three sets of rules shape what the product shows and stores: the DPDP Act and Rules, BCI Rule 36, and the line between information and legal advice. This is not legal advice; the PRD requires an advocate's written opinion before launch.

DPDP Act 2023 and Rules 2025

The PRD reads the data fiduciary duties as applying from about mid-May 2027 (Needs verification). Build to them now.

Requirement	What we build
Consent notice	Shown at sign-up: purpose, data collected, retention period, how to withdraw. Each acceptance saved in the consents table
Retention per data type	Account data, chats, uploads, audit log, backups and Langfuse logs each get a stated period. All Needs verification until counsel decides
User deletion	Delete a chat or upload any time; DELETE /me removes the account and its data, deletes the Firebase user and cancels the Razorpay subscription (payment and invoice records kept as the law requires; Needs counsel); GET /me/export gives users a copy of their data; backups age out on their normal cycle
Breach playbook	A written plan: notify affected users and the Data Protection Board, with a detailed report to the Board within 72 hours (EY summary; Needs verification against the Rules)
Roles	For account data, KanoonBuddy is the data fiduciary. For an advocate's uploaded client documents, the advocate is probably the fiduciary and KanoonBuddy a processor (Needs counsel). The terms of service must say so

Requirement	What we build
Data sent abroad	Voyage, Gemini and Jev process data outside India. Each vendor's terms and location checked and listed on the privacy page (Needs verification)

BCI Rule 36

- No listing, ranking, matching or recommending advocates. No "find a lawyer" feature.
- No named advocate testimonials or success stories. No payments to advocates per client referral.
- Share cards and stats pages are educational and never name an advocate.
- Whether naming an advocate advisor on the website counts as publicity: Needs counsel.

Not legal advice

- **Every answer, stats page and Act page** carries a short notice: KanoonBuddy gives information, not legal advice.
- **Stronger notice when Jev's safety check flags a personal-advice question:** the answer opens by saying it cannot advise on the user's own situation and suggests consulting an advocate.
- Outcome statistics stay descriptive; no predictions or win chances.

Security

- **Encrypted at rest:** EBS disks, S3 buckets and backups, with keys in AWS KMS.
- **Encrypted in transit:** HTTPS for every outside connection.
- **Secrets:** API keys and passwords in AWS Secrets Manager, never in the code or a file on the server.
- **Private Docker network:** only the website and the API are reachable from the internet. Qdrant, PostgreSQL and Redis are never exposed.
- **Dependency updates:** GitHub Dependabot flags and patches outdated libraries.
- **Independent security test** before public launch (launch gate condition 5). Cost Needs verification: get quotes.

Build order for a solo builder

Build the riskiest part first: search plus the citation verifier. If that does not give good answers, nothing else matters. Each step ends with something you can try.

1. **Data (your laptop).** Count Supreme Court points in Qdrant. Build kanoon_v1 (dense + BM25 sparse) for the Supreme Court and acts_india. Check 20 searches by hand in the Qdrant dashboard. Apply for Jev early access and email TypeSafe about no-training terms. Test re-embedding Supreme Court chunks with a short header (case name, court, year) in front, and keep it if search improves. Run the Jev cleaning job on Supreme Court dispositions, agree the pass rule with the advocate, and start the 50-judgment Supreme Court hand check. Evaluate upgrading to snapshot v2026.08.1, which fills source_url on 83.3% of rows (Competitor claim), against its download time.
2. **Pipeline in a notebook or script.** Question in, answer with verified citations out, including conversation memory, query rewrite, the answer cache and the invented-citation log. Wire the Jev adapter with its fallback, and compare each Jev use on and off. Wire LiteLLM, Voyage rerank and Langfuse. Run 50 real questions and record cost per answer.
3. **Backend API.** FastAPI routes for research chat (streaming), Act and section lookup, record view, stats, consents and account deletion. Tests for the verifier.
4. **Website.** Chat screen, citation ticks, judgment view with paragraph anchors, Act pages, the data-as-of date, the CC BY credit, the not-legal-advice notice and the consent notice at sign-up.
5. **Login, usage meter and workspaces.** Firebase sign-in, users table, monthly limits per plan, Chambers organisations and memberships (shared billing; each member's chats and uploads stay private in v1).
6. **AWS server.** EC2 in Mumbai. r7g is an ARM (Graviton) server, so first check every Docker image has an ARM build. Docker Compose on a private Docker network, restore the snapshot from S3 and then delete the snapshot file to free disk, HTTPS, encrypted disks, secrets in AWS Secrets Manager, daily backups, GitHub Actions deploy.
7. **Upload and chat.** S3 upload links, background OCR job, voyage-4 embeddings (free local model as the fallback), the shared user_documents collection filtered by user, delete controls. Built now, switched on for users only if the interviews and the legal opinion support it.
8. **Payments.** Razorpay subscriptions for Student, Pro and Chambers (minimum 3 seats), and the webhook.
9. **Admin page, test set and public pages.** The 200-question test set, run from the admin page. Free public Act pages and Supreme Court stats pages with sitemap and share cards; stats go live only after the 50-judgment Supreme Court hand check passes. Draft the first knowledge notes (law maps and coverage rules) and send them for advocate review. Write the breach playbook and the retention list.
10. **Private beta.** 20 students and 10 advocates, as in the PRD.

11. **Launch gate.** Public launch happens only if ALL of these are true at the end of the private beta:
    a. Verified-citation rate of 98% or higher on the 200-question test set.
    b. At least 6 of the 10 beta advocates say they would pay the Pro price.
    c. Zero confirmed fabricated citations reaching a beta user.
    d. The advocate's written legal opinion received.
    e. The independent security test passed.
    f. The data-as-of date shown prominently on every result, stats page and Act page. (Decision D2 replaces the PRD's "first data refresh complete".)

    If any condition fails, the launch date moves. Akshita Johari decides.

Solo vs team

The PRD plans 3 full-time engineers and estimates 18.5 person-weeks of engineering for weeks 3 to 10, plus design, bug fixing and security work. This plan is one person building with AI coding agents.

Setup	Rough time for steps 2 to 9	Label
PRD: 3 engineers	8 weeks (weeks 3 to 10)	PRD estimate
Solo with AI agents, full time	About 10 to 16 weeks	Estimate
Solo with AI agents, part time	Roughly double the full-time range	Estimate

Needs verification: time build steps 1 and 2, compare them with the plan, and replace this range with a measured one.

Not in the MVP build order

Item	Priority	Note
Hindi	P1 (within 3 months of launch)	P0 if the interviews show strong demand. See Hindi (P1)
Drafting, review and compare	P1 (within 3 months of launch)	Editor (TipTap) and Word export, as in the system architecture

Item	Priority	Note
Data refresh pipeline	Post-MVP backlog (decision D2)	MVP ships on snapshot v2026.08 with the data-as-of date. Later plan: scrape after the snapshot date, parse to the same schema, chunk, embed with voyage-4, add to Qdrant, reconcile counts, re-run the audit sample, publish a change log. Cadence and owner decided later
High Courts	Phase 2	When paying users cover the cost. See Phase 2: High Courts
Tribunals	Post-launch expansion	Free PDFs with no text layer, so the cost is OCR work. Not advertised at launch
Counter arguments, moot court, study buddy	P2	LangGraph only if needed

Risks

Risk	Why it matters	What we do
Jev is new	Early access only; servers on the US West Coast; data and no-training terms not confirmed; small startup	Own adapter, per-use switches, Gemini fallback, upload text never sent to Jev
One server, no failover	If the EC2 server fails, the whole product is down	EC2 automatic recovery, daily snapshots in S3, a written rebuild plan (target under 1 hour). Second server when paying users cover it
Memory pressure on one 16 GB server	Qdrant, PostgreSQL, Redis, website, backend, worker, backup reranker share it	Quantized vectors, worker capped at 1 CPU, watch memory in week 1; move up to 32 GB if needed

Risk	Why it matters	What we do
Voyage lock-in	All stored vectors need voyage-4 query embeddings; a price rise or outage hits search	Keep it behind one adapter; cache repeated questions; keyword search and Act lookup still work without it
Dependence on Vaquill	One organisation publishes the whole dataset	Keep the licence, dataset card and checksums as downloaded; keep an offline copy; ask Vaquill the PRD's three questions
High Court data quality	577 messy disposition values; 0% citation strings; junk judge and party names	Phase 2 only; cleaning job and hand check before High Court stats; hide junk fields
Frozen data	MVP ships without a refresh (decision D2), so recent judgments are missing	Data-as-of date on every result; refresh pipeline in the backlog
Solo build	Timeline may slip	Riskiest parts first (steps 1 and 2); re-plan from measured time

Open decisions

Decision	Options	Working choice	What settles it
Supreme Court point count and server size	r7g.large (16 GB) or r7g.xlarge (32 GB)	r7g.large	Count points in step 1; load-test search with quantization
Court field name in the Qdrant payload	Unknown	None	Inspect 5 Supreme Court points locally
AI model for each plan	Gemini Flash-Lite family, Claude, GPT, Kimi K3	Free: Gemini 2.5 Flash-Lite. Student and Pro: Gemini 3.1 Flash-Lite (both paid tier). A stronger model later for hard Pro questions	Week-1 test on 50 real questions: correctness, cost, speed

Decision	Options	Working choice	What settles it
Backup AI provider	OpenRouter, Groq or a second big vendor	Undecided	Check each provider's no-training terms
Re-ranker backup	Open-source bge-reranker on the server, or skip	bge-reranker	Test whether it fits in 16 GB alongside everything else
OCR quality for scans	Tesseract only, or Textract for hard scans	Tesseract only	Test 20 real court scans
Upload auto-delete period	7, 30 or 90 days	Undecided	Advocate's legal opinion
Hosting the website	Same EC2 server, or AWS Amplify	Same server	Move only if the server gets busy
GST in shown prices	Included or added	Undecided (from the PRD)	Accountant
Supabase or Cognito later	Stay on Firebase, or move	Stay	Only the triggers in Login and payments
Data-refresh method	Run the scrapers, or license daily sync from Vaquill. Post-MVP backlog (decision D2); MVP ships on the snapshot with a data-as-of date	Undecided (from the PRD)	Vaquill's reply on price and deltas
Re-embed Supreme Court chunks with a header	Keep Vaquill's vectors as they are, or re-embed with case name, court and year in front of each chunk	Keep as they are	Compare both on the test questions in build step 1; under ₹3,000 one-time (Estimate)
Answer cache lifetime	1, 7 or 30 days	7 days	Cache hit rate and user complaints in the beta
Upload size and page limits	For example 25 MB and 200 pages	Undecided	OCR time on real scans, and what interviewees upload

Decision	Options	Working choice	What settles it
Jev thresholds	Gate starts at 0.6; claim support and scope still to set	0.6 for the gate	200-question test set, Jev on vs off
Send upload text to Jev	Decided: never. Upload chats use Gemini (paid tier) for these checks	Gemini only	TypeSafe's written no-training and retention terms
Order to switch on Jev uses	All at launch, or one at a time	Offline cleaning first, then gate and claim support, then the rest	Test results and early-access availability
Who reviews knowledge notes	The advocate advisor, or a paid one-off reviewer	Advocate advisor	Quote for reviewing the law maps
Which notes beyond the first set	Grow by topic, or only where answers go wrong	Only where answers go wrong	Wrong-answer reports and test-set failures
Upload embedding model	voyage-4 first, or a free local model (for example bge-m3)	voyage-4, local model as fallback	Voyage's terms for upload text, the legal opinion, and quality on 20 real uploads
50-judgment Supreme Court pass rule	How many must match for stats to go live	Undecided	Agreed with the advocate before checking
Phase 2 court order	Bombay, Telangana, Kerala first, or Delhi first for practical demand	Bombay, Telangana, Kerala	The 20 interviews
Final product name	KanoonBuddy (working name) or Praman (PRD's pick)	Undecided	Trademark search with IP India
Voyage free allowance	200M free tokens once, or every month	Assume once	Voyage's pricing page or support

Decision	Options	Working choice	What settles it
Snapshot v2026.08.1	Upgrade from v2026.08 (source_url on 83.3% of rows), or stay	Undecided	Build step 1 evaluation against download time
What DELETE /me keeps	Delete everything, or keep payment and invoice records the law requires	Keep invoices only	Counsel
When to switch on model routing	At launch, or once a stronger model is chosen	Off at launch	The week-1 test of 50 real questions

Glossary in plain words

Every tool and term in this doc, explained without jargon.

The building blocks

Term	What it is	Its job in KanoonBuddy
Firebase	A Google service for login	The security guard at the door: shows sign-in, stores passwords, confirms who each user is. Knows nothing about what they do inside
PostgreSQL	A database: organised tables, like very strict Excel sheets	Holds users, plans, usage counts, chats, upload records and the Supreme Court text
Qdrant	A search database for meaning	Finds judgments and sections that match the idea of a question, not just its words
Redis	Very fast short-term memory	Holds the waiting list of upload jobs, and saved answers to repeated questions
Amazon S3	Online file storage, like a private Google Drive for apps	Stores uploaded files and backups, encrypted

Term	What it is	Its job in KanoonBuddy
AWS EC2	A computer you rent in Amazon's data centre	The one server in Mumbai that runs everything
Docker / Docker Compose	Sealed boxes that each hold one program with everything it needs	Lets the whole system start with one command on any computer, and move easily later
Next.js	A tool for building websites	The website users see on phone and laptop
FastAPI	A tool for building a backend in Python	The "kitchen" behind the website: takes questions, runs the pipeline, sends answers
Python / TypeScript	Programming languages	Python for the backend and AI work; TypeScript for the website
Razorpay	An Indian payment service	Takes UPI and card payments for Student, Pro and Chambers plans
Voyage AI	A company that turns text into numbers that capture meaning	Makes search by meaning possible, and picks the best passages (re-ranking)
LiteLLM	One plug that fits many AI models	Lets you switch between Gemini, Claude, GPT and others by changing one setting
Gemini Flash-Lite	A cheap, fast Google AI model	Writes the answers in v1
Tesseract	Free software that reads text from images	Reads scanned court documents (OCR)
Sentry	An alarm for crashes	Tells you when something breaks, and where
Langfuse	A logbook for AI calls	Records every question, answer, cost and speed so you can see what is happening
GitHub / GitHub Actions	Where your code is stored / a robot that tests and deploys it	Every change is tested, then put on the server automatically
Jev (TypeSafe AI)	An AI that picks one answer from a fixed list, with a confidence score, instead of writing text	The quick referee: checks if a question is in scope, if passages answer it, and if each citation really supports its sentence

The ideas

Term	Plain meaning
Frontend / backend	The part users see (website) / the part that does the work behind it
API	The menu of requests the website can make to the backend, such as "ask a question" or "upload a file"
Pipeline	A fixed list of steps every question goes through, always in the same order
Agent	An AI that decides its own next step. Used here to help write code, not to answer users
Embedding / vector	A list of numbers that stands for a passage's meaning, so similar meanings sit close together
Semantic search	Search by meaning: "bail for money crimes" finds "economic offences"
Keyword search / BM25	Search by exact words: finds "Section 438" exactly
Hybrid search	Running both searches together and merging the results
Re-ranking	A second, more careful pass that puts the best few passages on top
Chunk	A short piece of a long judgment, because a whole judgment is too long to send to the AI
Citation verifier	The checker that removes any case the AI cites that was not actually found, or that does not support what the AI said
OCR	Reading text out of a scanned image of a page
Token	A piece of a word. AI costs are counted in tokens: roughly 1,000 tokens is 750 English words
Cache	A saved copy of an answer, so a repeated question costs nothing
Streaming / SSE	The answer appears word by word instead of all at once
Pre-signed URL	A one-time upload link that lets the browser send a file straight to S3

Term	Plain meaning
Token (login)	A digital pass Firebase gives a logged-in user; the backend checks it on every request
Adapter	One file that talks to one outside service, so it can be swapped without touching the rest
Environment variable / config	Settings kept outside the code, such as API keys and the model name
Migration	A numbered file that records each change to the database tables, so changes can be undone
Payload	The labels stored with each item in Qdrant, such as court, year and section number, used as filters
Quantization	Squeezing the vectors so they use about a quarter of the memory, with little loss in search quality
Audit log	A record of who did what and when (logins, uploads, deletions)
Uptime / 99.5%	The share of time the app is working; 99.5% allows about 3.6 hours of downtime a month
Calibrated confidence	A confidence score you can trust: when it says 80% sure, it is right about 80% of the time
Threshold	The cut-off score: above it, the app goes ahead; below it, the app refuses or shows a warning
Fallback	The backup the app uses automatically when a service is down or too slow
OKF (Open Knowledge Format)	Google's standard for giving AI checked knowledge as linked Markdown files, like a small wiki
RAG	Search first, then let the AI answer using only what was found
SEO / sitemap	Making pages easy for Google to find / a list of every public page, handed to Google
Share card	The preview picture that appears when a link is shared on WhatsApp or LinkedIn
DPDP	India's Digital Personal Data Protection Act 2023 and its Rules: how personal data must be collected, kept and deleted

Term	Plain meaning
Data fiduciary / processor	The one who decides why data is used and is responsible for it / the one who only handles it on their behalf
Launch gate	A checklist that must all pass before the public launch; if any fails, the date moves

Change log (9 October 2026)

Revision after the full PRD review, applying final decisions D1 to D6. Nothing was removed unless it contradicted the PRD or a decision.

- **A. Scope:** v1 stays Supreme Court + acts_india. New section Phase 2: High Courts (Bombay, Telangana, Kerala) with the sizing method, an illustrative cost and the PRD's trigger.
- **B. Outcome statistics and public pages:** new section Public pages and outcome statistics; GET /stats and /sitemap.xml routes; outcome_label and is_final_outcome in the cases table and Qdrant payload; stats gated on the 50-judgment Supreme Court hand check (D1); website box in the diagram updated.
- **C. Keyword search:** Qdrant BM25 marked decided, settling the PRD's open choice.
- **D. Hindi:** new section Hindi (P1) with a test plan (D3).
- **E. Launch gate:** build step 11 with six conditions; condition 6 is "data-as-of date shown prominently" instead of "first data refresh complete" (D2, D5).
- **F. Team and timeline:** Solo vs team note; solo range is an Estimate pending steps 1 and 2.
- **G. Upload and chat:** built at step 7, shipped only if interviews and the legal opinion support it. Embedding choice compared; voyage-4 first with a free local model as the fallback (changed in the second pass, C2).
- **H. Citation verifier:** added the PRD's "what the tick does not mean" table and badge wording; Jev claim check labelled as beyond the PRD.
- **I. Data model:** 13 → 17 tables (organisations, memberships, consents, court_coverage; renamed in the second pass, C8); data_as_of per court; outcome fields.
- **J. Data refresh:** moved to the post-MVP backlog per decision D2, which overrides the request to add it as a build step. The PRD's incremental plan is kept in the backlog table.
- **K. Compliance:** new section covering DPDP, BCI Rule 36 and the not-legal-advice notice on every answer (stronger when Jev flags personal advice); CC BY credit text fixed in the Show step (D4, D6).

- **L. Risks:** new Risks section.
- **M. Cleanups:** labels aligned with the PRD (Measured, Dataset card, Competitor claim added); naming note (Praman; internal names easy to rename); owner named as Akshita Johari; revision date added. No old names (Drex) were found.

Second pass (9 October 2026)

Fixes contradictions and arithmetic, and adds what v1 was missing. Where this list conflicted with decisions D1 and D2, those decisions were kept.

- **A1–A3:** AI model rows now match the tiers table (Free: Gemini 2.5 Flash-Lite; Student and Pro: Gemini 3.1 Flash-Lite; stronger model later). Model routing off at launch. Passage counts consistent: 50 → 10 → at most 8.
- **A4:** Citation verifier rewritten as four numbered checks; Supreme Court citation-string rule is now a display note.
- **A5:** Voyage about ₹0.05 per answer (re-rank about ₹0.044, rounded up to include the query embedding); scale row ₹1,500; question added on whether the 200M free tokens are one-time or monthly.
- **A6–A8:** Disposition cleaning covers only the Supreme Court; leading-cases note uses a citation-extraction script (Needs verification); the Growth column is now "Illustrative scale: 2,000 users", not a forecast.
- **B1–B4:** Launch collection 42 to 55 GB; memory about 4 to 5 GB; launch disk 200 GB (snapshot file deleted after restore); launch total about ₹7,200 to ₹7,500 a month, about 21 paying users.
- **C1–C2:** Upload text never reaches Jev. Uploads use voyage-4 first, terms Needs verification, local model as fallback.
- **C3:** New section Where data goes, with Langfuse masking and Sentry scrubbing.
- **C4:** act_status in the payload; in-force shown only when act_status is filled.
- **C5:** Outcome fields kept as outcome_label and is_final_outcome; stats stay in v1 per decision D1 (not moved after launch).
- **C6:** Not applied: data refresh stays in the post-MVP backlog per decision D2. court_coverage already exists.
- **C7:** Build step 1 evaluates snapshot v2026.08.1.
- **C8:** Chambers uses organisations and memberships; each member's chats and uploads stay private in v1.
- **C9–C10:** Export route, DELETE /me details (Firebase user, Razorpay subscription, invoices kept; Needs counsel); Security subsection.
- **C11–C14:** Mobile and slow networks section; Hindi tokenizer not a v1 blocker; ARM build check in step 6; product and collection names in config.

- **D:** Monthly limits on every plan; cache hits free; four-reason invented-citation metric stated.
- **E:** Four new Open decisions rows. No refresh build step added (D2).
- **Tables:** stays at 17, not 18, because the refresh-run table from C6 is not added.
- **Architecture:** Langfuse and Sentry shown as monitoring services.